BÀI THỰC HÀNH CTDL KIỂU LINKEDLIST

🎯 Mục đích

- **Hiểu bản chất của LinkedList:** Nắm vững khái niệm về nút (node), con trỏ `next` và `prev`.
- **Thành thạo các thao tác cơ bản:** Biết cách thêm, xóa các phần tử ở đầu, cuối và một vị trí bất kỳ trong danh sách.
- **Làm việc trực tiếp với Node:** Hiểu và thao tác trực tiếp với các đối tượng `Node` để thực hiện các thao tác phức tạp như chèn vào giữa hoặc duyệt danh sách theo cả hai chiều.
- **Phân tích hiệu năng:** Nhận biết được các kịch bản mà LinkedList vượt trội hơn Dynamic Array / List (ví dụ: thêm/xóa ở đầu hoặc giữa danh sách khi đã có con trỏ) và ngược lại.

📋 Yêu cầu chung cho tất cả bài tập

- **Môi trường:** Viết ứng dụng dạng Console Application bằng **Python**.
- **Code sạch:** Viết code rõ ràng, dễ đọc, có chú thích khi cần thiết.
- **Tương tác:** Cho phép người dùng nhập liệu để điều khiển chương trình.
- **Tư duy phản biện:** Tích cực sử dụng các công cụ AI không chỉ để giải bài tập, mà còn để đặt câu hỏi "Tại sao?" và "Nếu... thì sao?" để đào sâu kiến thức.

Bài tập 1: Xây dựng một Playlist nhạc - Mức độ: Dễ

Bài tập này giúp bạn làm quen với việc tạo LinkedList và các thao tác thêm vào cơ bản nhất.

Yêu cầu chi tiết:

Tạo một LinkedList để lưu tên các bài hát trong một playlist.

Viết chức năng cho phép người dùng:

1. Thêm một bài hát vào cuối playlist.
2. Thêm một bài hát vào đầu playlist (để phát ngay sau bài hiện tại).
3. Hiển thị playlist hiện tại.
4. Thoát.

Chương trình lặp lại menu cho đến khi người dùng chọn thoát.

💡 Gợi ý:

- Trong Python, ta có thể dùng thư viện chuẩn `collections.deque` (được hiện thực bằng danh sách liên kết đôi tối ưu thao tác ở hai đầu):

- Khởi tạo: `deque()`.
- Thêm vào cuối: `.append()` (tương đương `.AddLast()`).
- Thêm vào đầu: `.appendleft()` (tương đương `.AddFirst()`).
- Duyệt danh sách bằng vòng lặp `for song in playlist:`.

💻 Code mẫu (Python):

Python

```
from collections import deque

def main():
    playlist = deque()
    playlist.append("Shape of You")
    playlist.append("Blinding Lights")

    while True:
        print("\n--- PLAYLIST CỦA BẠN ---")
        print("1. Thêm bài hát vào cuối")
        print("2. Thêm bài hát vào đầu")
        print("3. Hiển thị playlist")
        print("4. Thoát")

        choice = input("Lựa chọn: ").strip()

        if choice == "1":
            song_end = input("Nhập tên bài hát: ").strip()
            playlist.append(song_end)
        elif choice == "2":
            song_first = input("Nhập tên bài hát: ").strip()
            playlist.appendleft(song_first)
        elif choice == "3":
            print("\n--- Đang phát ---")
            for song in playlist:
                print(f"- {song}")
        elif choice == "4":
            print("Đã thoát chương trình.")
            break
        else:
            print("Lựa chọn không hợp lệ.")

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Hỏi về bản chất:** Hỏi AI: *"LinkedList lưu trữ dữ liệu trong bộ nhớ khác gì so với List? Vẽ sơ đồ minh họa."*
- **Khám phá thêm:** Hỏi AI: *"Nếu tôi muốn thêm một bài hát ngay sau bài 'Shape of You', tôi phải làm thế nào?" (Điều này sẽ dẫn dắt bạn đến khái niệm LinkedListNode và AddAfter).*

Bài tập 2: Mô phỏng hàng chờ khám bệnh - Mức độ: Dễ - Trung bình

Bài tập này mô phỏng hoạt động của một hàng đợi (Queue), một ứng dụng kinh điển của LinkedList theo nguyên tắc "Vào trước - Ra trước" (FIFO).

Yêu cầu chi tiết:

Tạo một LinkedList để lưu tên các bệnh nhân đang chờ khám.

Xây dựng menu chức năng:

1. Thêm bệnh nhân mới vào hàng chờ.
2. Gọi bệnh nhân tiếp theo vào khám (lấy ra người đầu tiên trong hàng).
3. Xem danh sách bệnh nhân đang chờ.
4. Thoát.

Nếu không còn ai trong hàng chờ mà bác sĩ vẫn gọi, hãy thông báo "Hàng chờ trống."

💡 Gợi ý:

- Thêm bệnh nhân mới vào cuối hàng chờ (`.append()`).
- Khi gọi bệnh nhân, lấy ra người đầu tiên và xóa khỏi hàng chờ (`.popleft()`, tương đương `.RemoveFirst()` trong C#).
- Luôn kiểm tra `len(patient_queue) > 0` trước khi cố gắng gọi bệnh nhân.

💻 Code mẫu (Python):

Python

```
from collections import deque

def main():
    patient_queue = deque()

    # Logic thêm bệnh nhân
    # patient_queue.append("Nguyễn Văn A")

    # Logic gọi bệnh nhân
    if len(patient_queue) > 0:
        next_patient = patient_queue.popleft()
        print(f"Mời bệnh nhân {next_patient} vào khám!")
    else:
        print("Hàng chờ trống.")

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **So sánh hiệu năng:** Hỏi AI: *"Tại sao dùng LinkedList / deque cho hàng chờ lại hiệu quả hơn Python list thông thường? Điều gì xảy ra nếu tôi dùng list.pop(0) liên tục?" (AI sẽ giải thích về chi phí $O(n)$ dịch chuyển phần tử trong mảng).*
- **Tìm hiểu cấu trúc dữ liệu:** Hỏi AI: *"Ngoài LinkedList, Python có module queue (như queue.Queue) chuyên dụng. Nó có ưu điểm gì so với việc tự triển khai bằng LinkedList?"*

Bài tập 3: Mô phỏng lịch sử trình duyệt web - Mức độ: Trung bình

Bài tập này yêu cầu bạn làm việc trực tiếp với các Node để di chuyển tới lui trong danh sách, một thế mạnh của LinkedList.

Yêu cầu chi tiết:

- Tạo cấu trúc Doubly LinkedList (Danh sách liên kết đôi) với các nút có liên kết `prev` và `next`.
- Tạo một biến `current_node` để theo dõi trang web người dùng đang xem.
- Xây dựng menu chức năng:
    1. **Truy cập một trang mới (Visit):** Người dùng nhập URL. URL này sẽ được thêm vào ngay sau `current_node`. Tất cả các trang "phía trước" (forward history) sẽ bị cắt bỏ. `current_node` được cập nhật thành trang mới.
    2. **Quay lại (Back):** Di chuyển `current_node` về nút phía trước (`prev`). Nếu đang ở trang đầu tiên thì không di chuyển được.
    3. **Tới trang trước (Forward):** Di chuyển `current_node` tới nút phía sau (`next`). Nếu đang ở trang cuối cùng thì không di chuyển được.
    4. Hiển thị trang hiện tại.

💡 Gợi ý:

- Khi Visit một trang mới, ngắt liên kết `current_node.next = None` để loại bỏ lịch sử phía trước.
- Tạo nút mới, gán `current_node.next = new_node`, `new_node.prev = current_node` và trỏ `current_node = new_node`.
- Luôn kiểm tra `current_node.prev is not None` trước khi Back và `current_node.next is not None` trước khi Forward.

💻 Code mẫu (Python):

Python

```
class Node:
    def __init__(self, value: str):
        self.value = value
        self.prev = None
```

```
        self.next = None

class BrowserHistory:
    def __init__(self, homepage: str):
        self.current_node = Node(homepage)

    def visit(self, url: str):
        # Cắt bỏ lịch sử forward phía sau
        new_node = Node(url)
        self.current_node.next = new_node
        new_node.prev = self.current_node
        self.current_node = new_node
        print(f"Đã truy cập: {self.current_node.value}")

    def go_back(self):
        if self.current_node.prev is not None:
            self.current_node = self.current_node.prev
            print(f"Quay lại trang: {self.current_node.value}")
        else:
            print("Không có trang trước đó.")

    def go_forward(self):
        if self.current_node.next is not None:
            self.current_node = self.current_node.next
            print(f"Đi tới trang: {self.current_node.value}")
        else:
            print("Không có trang phía trước.")

def main():
    # Bắt đầu bằng cách truy cập trang google.com
    browser = BrowserHistory("google.com")

    # Minh họa chức năng Visit trang mới
    browser.visit("vnexpress.net")

    # Minh họa chức năng Back
    browser.go_back()

    # Minh họa chức năng Forward
    browser.go_forward()

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Gỡ lỗi logic:** Sao chép code của bạn (dù đúng hay sai) và hỏi AI: *"Tôi đang viết chương trình mô phỏng lịch sử trình duyệt bằng Doubly LinkedList trong Python. Logic của tôi có vấn đề gì không? Có trường hợp nào tôi chưa xử lý không?"*
- **Trực quan hóa:** Yêu cầu AI: *"Hãy vẽ sơ đồ các node và con trỏ prev, next của LinkedList trong bài toán lịch sử trình duyệt sau các bước: Visit A, Visit B, Back, Visit C."*

Bài tập 4: Trò chơi đếm số và loại người (Josephus Problem) - Mức độ: Khó

Đây là một bài toán kinh điển, và LinkedList là một cấu trúc dữ liệu rất phù hợp để giải quyết vì nó cho phép xóa một phần tử ở giữa rất hiệu quả.

Yêu cầu chi tiết:

- Cho n người đứng thành một vòng tròn, được đánh số từ 1 đến n.
- Bắt đầu từ người số 1, đếm theo chiều kim đồng hồ k người. Người thứ k sẽ bị loại khỏi vòng tròn.
- Tiếp tục đếm từ người tiếp theo người vừa bị loại, lại đếm k người và loại người thứ k.
- Quá trình tiếp tục cho đến khi chỉ còn lại một người. Hãy tìm ra người cuối cùng đó là ai.
- Viết chương trình cho phép người dùng nhập n và k, sau đó in ra số thứ tự của người cuối cùng còn lại.

💡 Gợi ý:

- Tạo danh sách liên kết vòng (Circular LinkedList) hoặc dùng danh sách liên kết đôi trỏ vòng từ `tail.next = head` và `head.prev = tail`.
- Di chuyển con trỏ đi k-1 bước.
- Nút tại vị trí này chính là nút cần xóa: cập nhật liên kết của nút trước và sau nó, sau đó chuyển con trỏ hiện tại sang nút kế tiếp để tiếp tục vòng đếm.

💻 Code mẫu (Python):

Python

```
class Node:
    def __init__(self, value: int):
        self.value = value
        self.next = None
        self.prev = None

def solve_josephus(n: int, k: int):
    # Khởi tạo Circular Doubly LinkedList gồm n nút
    head = Node(1)
    prev = head
    for i in range(2, n + 1):
        curr = Node(i)
        prev.next = curr
        curr.prev = prev
        prev = curr

    # Nối vòng tròn (head <-> tail)
    prev.next = head
    head.prev = prev

    current = head
    remaining = n
```

```
    while remaining > 1:
        # Di chuyển k - 1 bước
        for _ in range(k - 1):
            current = current.next

        # Node cần xóa là current
        node_to_remove = current
        next_node = current.next

        # Cắt bỏ node_to_remove khỏi vòng tròn
        node_to_remove.prev.next = node_to_remove.next
        node_to_remove.next.prev = node_to_remove.prev

        remaining -= 1
        print(f"Đã loại người số: {node_to_remove.value}. Còn lại:
{remaining} người.")

        # Bắt đầu vòng đếm mới từ nút tiếp theo
        current = next_node

    print(f"\nNgười sống sót cuối cùng là: {current.value}")

def main():
    n = int(input("Nhập số người (n): "))
    k = int(input("Nhập bước đếm (k): "))
    solve_josephus(n, k)

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Tối ưu hóa:** Hỏi AI: *"Có thể giải bài toán Josephus bằng công thức truy hồi toán học thay vì mô phỏng bằng LinkedList không? Độ phức tạp sẽ giảm như thế nào?"*
- **Phân tích thuật toán:** Hỏi AI: *"Giải thích độ phức tạp thời gian của thuật toán Josephus khi giải bằng LinkedList và so sánh nó với việc giải bằng mảng thông thường (Python list)."*

Bài tập 5: Sắp xếp lại một Playlist nhạc - Mức độ: Rất khó

Bài tập này kiểm tra kỹ năng thao tác node ở mức độ cao: cắt một đoạn của LinkedList và ghép nó vào một vị trí khác.

Yêu cầu chi tiết:

- Tạo một LinkedList chứa một playlist nhạc có sẵn (ít nhất 8-10 bài hát).
- Hiển thị playlist ban đầu.
- Yêu cầu người dùng nhập vào:
    - Tên bài hát bắt đầu của đoạn cần di chuyển (`start_song`).
    - Tên bài hát kết thúc của đoạn cần di chuyển (`end_song`).

    - Tên bài hát mà đoạn nhạc trên sẽ được chèn vào ngay sau nó (`destination_song`).
- Thực hiện việc cắt đoạn playlist từ `start_song` đến `end_song` và chèn nó vào sau `destination_song`.
- Hiển thị playlist sau khi đã sắp xếp lại. Xử lý các trường hợp nhập không hợp lệ.

💡 Gợi ý:

- Tìm các nút: `start_node`, `end_node`, `dest_node`.
- Lấy các nút từ `start_node` đến `end_node` đưa vào danh sách tạm.
- Cắt rời đoạn này khỏi danh sách gốc bằng cách nối lại nút trước `start_node` với nút sau `end_node`.
- Nối lại đoạn đã cắt vào ngay sau `dest_node`.

💻 Code mẫu (Python - Thao tác trực tiếp trên con trỏ):

Python

```
class Node:
    def __init__(self, value: str):
        self.value = value
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self, items=None):
        self.head = None
        self.tail = None
        if items:
            for item in items:
                self.append(item)

    def append(self, value: str):
        new_node = Node(value)
        if not self.head:
            self.head = self.tail = new_node
        else:
            self.tail.next = new_node
            new_node.prev = self.tail
            self.tail = new_node

    def find(self, value: str):
        curr = self.head
        while curr:
            if curr.value == value:
                return curr
            curr = curr.next
        return None

    def to_list(self):
        res = []
        curr = self.head
        while curr:
```

```
            res.append(curr.value)
            curr = curr.next
        return res

def move_segment(dll: DoublyLinkedList, start_val: str, end_val: str,
dest_val: str):
    start_node = dll.find(start_val)
    end_node = dll.find(end_val)
    dest_node = dll.find(dest_val)

    if not start_node or not end_node or not dest_node:
        print("Tên bài hát không hợp lệ!")
        return

    # Kiểm tra end_node phải nằm sau hoặc cùng là start_node
    curr = start_node
    is_valid_range = False
    while curr:
        if curr == end_node:
            is_valid_range = True
            break
        curr = curr.next
    if not is_valid_range:
        print("Đoạn cần cắt không hợp lệ (start_song phải xuất hiện trước
end_song).")
        return

    # Kiểm tra dest_node không được nằm bên trong đoạn cần cắt
    curr = start_node
    while True:
        if curr == dest_node:
            print("Vị trí đích không được nằm trong đoạn cần di chuyển!")
            return
        if curr == end_node:
            break
        curr = curr.next

    # Bước 1: Cắt đoạn [start_node -> end_node] ra khỏi danh sách
    before_start = start_node.prev
    after_end = end_node.next

    if before_start:
        before_start.next = after_end
    else:
        dll.head = after_end

    if after_end:
        after_end.prev = before_start
    else:
        dll.tail = before_start

    # Bước 2: Chèn đoạn vừa cắt vào sau dest_node
    after_dest = dest_node.next

    dest_node.next = start_node
    start_node.prev = dest_node
```

```
    end_node.next = after_dest
    if after_dest:
        after_dest.prev = end_node
    else:
        dll.tail = end_node

def main():
    # Khởi tạo playlist ban đầu: A -> B -> C -> D -> E -> F -> G -> H
    playlist = DoublyLinkedList(["A", "B", "C", "D", "E", "F", "G", "H"])
    print("Playlist ban đầu:", " -> ".join(playlist.to_list()))

    # Yêu cầu: Di chuyển đoạn "C", "D", "E" vào sau "G"
    # Kết quả mong muốn: A -> B -> F -> G -> C -> D -> E -> H
    move_segment(playlist, start_val="C", end_val="E", dest_val="G")
    print("Playlist sau khi sắp xếp:", " -> ".join(playlist.to_list()))

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI để gia tăng trải nghiệm:

- **Tìm giải pháp tốt hơn:** Hỏi AI: *"Trong code nối con trỏ trực tiếp, hãy phân tích độ phức tạp thời gian và không gian so với việc sao chép các node sang một mảng tạm thời."*
- **Xử lý lỗi:** Thảo luận với AI: *"Hãy liệt kê tất cả các trường hợp biên (edge cases) có thể làm crash hàm `move_segment` (ví dụ: cắt nút đầu tiên, cắt nút cuối cùng, hoặc chèn vào nút cuối cùng của danh sách) và kiểm tra xem đoạn code trên đã xử lý hết chưa."*

BÀI THỰC HÀNH NÂNG CAO: CÀI ĐẶT THƯ VIỆN `my_linkedlist`

🎯 Mục tiêu

1. Tự tay thiết kế và đóng gói cấu trúc dữ liệu **Doubly Linked List (Danh sách liên kết đôi)** thành một module Python có thể tái sử dụng (`import`).
2. Hiện thực hóa các thao tác cơ bản và nâng cao với độ phức tạp tối ưu:
   - Thêm/xóa đầu và cuối: $O(1)$.
   - Tìm kiếm: $O(n)$.
   - Chèn/xóa khi đã biết nút: $O(1)$.
3. Tích hợp các **Python Dunder (Double underscore) Methods** (`__len__`, `__iter__`, `__repr__`, `__bool__`) để cấu trúc dữ liệu hoạt động tự nhiên như kiểu dữ liệu nguyên bản của Python (duyệt `for ... in`, đo độ dài `len()`, kiểm tra rỗng `if list:`).

🗀 Cấu trúc thư mục dự án

Plaintext
```
linked_list_project/
│
├── my_linkedlist.py       # Module thư viện bạn tự xây dựng
├── main_test.py           # File kiểm thử thư viện
└── bai_tap_playlist.py    # Ví dụ import vào giải bài tập Playlist
```

💻 Mã nguồn thư viện: File `my_linkedlist.py`

Hãy tạo file `my_linkedlist.py` và hoàn thiện đoạn mã sau:

Python
```
"""
Module: my_linkedlist
Mô tả: Tự cài đặt Doubly Linked List (Danh sách liên kết đôi)
       để phục vụ việc học tập và thực hành CTDL & GT.
"""
from typing import Any, Optional, Iterator


class Node:
    """Đại diện cho một nút (Node) trong Danh sách liên kết đôi."""

    def __init__(self, data: Any):
        self.data: Any = data
        self.prev: Optional['Node'] = None
        self.next: Optional['Node'] = None

    def __repr__(self) -> str:
```

```
        return f"Node({self.data})"


class DoublyLinkedList:
    """Danh sách liên kết đôi hỗ trợ quản lý nút đầu (head) và đuôi
(tail)."""

    def __init__(self, iterable=None):
        self.head: Optional[Node] = None
        self.tail: Optional[Node] = None
        self._size: int = 0

        # Nếu truyền vào một iterable (list, tuple...), nạp sẵn dữ liệu
        if iterable is not None:
            for item in iterable:
                self.add_last(item)

    # ================= CÁC DUNDER METHODS HỖ TRỢ PYTHON ================
    def __len__(self) -> int:
        """Cho phép dùng hàm len(danh_sach) với O(1)."""
        return self._size

    def __bool__(self) -> bool:
        """Cho phép kiểm tra 'if danh_sach:' để biết danh sách có rỗng hay
không."""
        return self._size > 0

    def __iter__(self) -> Iterator[Any]:
        """Cho phép duyệt danh sách bằng vòng lặp 'for item in
danh_sach:'."""
        curr = self.head
        while curr:
            yield curr.data
            curr = curr.next

    def __repr__(self) -> str:
        """Hiển thị danh sách trực quan dạng: A <-> B <-> C."""
        nodes = []
        curr = self.head
        while curr:
            nodes.append(str(curr.data))
            curr = curr.next
        return " <-> ".join(nodes) if nodes else "Empty LinkedList"

    # ================= CÁC THAO TÁC CƠ BẢN ================
    @property
    def first(self) -> Optional[Node]:
        """Lấy nút đầu tiên (O(1))."""
        return self.head

    @property
    def last(self) -> Optional[Node]:
        """Lấy nút cuối cùng (O(1))."""
        return self.tail

    def add_first(self, data: Any) -> Node:
        """Thêm một phần tử vào đầu danh sách (O(1))."""
```

```
        new_node = Node(data)
        if self.head is None:
            self.head = self.tail = new_node
        else:
            new_node.next = self.head
            self.head.prev = new_node
            self.head = new_node
        self._size += 1
        return new_node

    def add_last(self, data: Any) -> Node:
        """Thêm một phần tử vào cuối danh sách (O(1))."""
        new_node = Node(data)
        if self.tail is None:
            self.head = self.tail = new_node
        else:
            self.tail.next = new_node
            new_node.prev = self.tail
            self.tail = new_node
        self._size += 1
        return new_node

    def add_after(self, node: Node, data: Any) -> Node:
        """Chèn một giá trị ngay sau nút được chỉ định (O(1))."""
        if node is None:
            raise ValueError("Node tham chiếu không được là None.")

        if node == self.tail:
            return self.add_last(data)

        new_node = Node(data)
        new_node.prev = node
        new_node.next = node.next
        node.next.prev = new_node
        node.next = new_node
        self._size += 1
        return new_node

    def remove_first(self) -> Any:
        """Xóa phần tử đầu danh sách và trả về giá trị (O(1))."""
        if self.head is None:
            raise IndexError("Không thể xóa từ danh sách rỗng.")

        val = self.head.data
        if self.head == self.tail:
            self.head = self.tail = None
        else:
            self.head = self.head.next
            self.head.prev = None
        self._size -= 1
        return val

    def remove_last(self) -> Any:
        """Xóa phần tử cuối danh sách và trả về giá trị (O(1))."""
        if self.tail is None:
            raise IndexError("Không thể xóa từ danh sách rỗng.")
```

```
        val = self.tail.data
        if self.head == self.tail:
            self.head = self.tail = None
        else:
            self.tail = self.tail.prev
            self.tail.next = None
        self._size -= 1
        return val

    def remove(self, node: Node) -> None:
        """Xóa trực tiếp một Node ra khỏi danh sách (O(1))."""
        if node is None:
            return

        if node == self.head:
            self.remove_first()
            return
        if node == self.tail:
            self.remove_last()
            return

        node.prev.next = node.next
        node.next.prev = node.prev
        node.next = None
        node.prev = None
        self._size -= 1

    def find(self, data: Any) -> Optional[Node]:
        """Tìm và trả về Node đầu tiên chứa giá trị data (O(n))."""
        curr = self.head
        while curr:
            if curr.data == data:
                return curr
            curr = curr.next
        return None

    def clear(self) -> None:
        """Xóa toàn bộ danh sách."""
        self.head = None
        self.tail = None
        self._size = 0
```

□ File kiểm thử thư viện: `main_test.py`

Tạo file `main_test.py` cùng thư mục để kiểm tra tất cả các hàm:

Python

```
from my_linkedlist import DoublyLinkedList

def run_tests():
    print("--- 1. Kiểm tra khởi tạo & hiển thị ---")
    dll = DoublyLinkedList(["A", "B", "C"])
    print(f"Danh sách ban đầu: {dll}")
    print(f"Độ dài: {len(dll)}")  # Test __len__
```

```
    print("\n--- 2. Kiểm tra thêm đầu / cuối ---")
    dll.add_first("START")
    dll.add_last("END")
    print(f"Sau khi thêm: {dll}")

    print("\n--- 3. Kiểm tra duyệt vòng lặp (Iterator) ---")
    for item in dll:
        print(f"-> {item}")

    print("\n--- 4. Kiểm tra tìm kiếm và chèn giữa ---")
    node_b = dll.find("B")
    if node_b:
        dll.add_after(node_b, "B_PLUS")
        print(f"Sau khi chèn B_PLUS sau B: {dll}")

    print("\n--- 5. Kiểm tra xóa ---")
    print("Xóa đầu:", dll.remove_first())
    print("Xóa cuối:", dll.remove_last())
    node_bp = dll.find("B_PLUS")
    if node_bp:
        dll.remove(node_bp)
        print("Đã xóa nút B_PLUS")
    print(f"Danh sách hiện tại: {dll}")

if __name__ == "__main__":
    run_tests()
```

💡 Ví dụ áp dụng: Giải quyết Bài tập Playlist (`bai_tap_playlist.py`)

Khi bạn đã có `my_linkedlist.py`, việc code các bài tập khác trở nên cực kỳ ngắn gọn và sạch sẽ:

Python
```
from my_linkedlist import DoublyLinkedList

def main():
    # Import và sử dụng thư viện tự viết
    playlist = DoublyLinkedList(["Shape of You", "Blinding Lights"])

    while True:
        print("\n--- PLAYLIST CỦA BẠN ---")
        print("1. Thêm bài hát vào cuối")
        print("2. Thêm bài hát vào đầu")
        print("3. Chèn bài hát sau một bài cụ thể")
        print("4. Hiển thị playlist")
        print("5. Thoát")

        choice = input("Lựa chọn: ").strip()

        if choice == "1":
            song = input("Nhập tên bài hát: ").strip()
            playlist.add_last(song)
```

```
        elif choice == "2":
            song = input("Nhập tên bài hát: ").strip()
            playlist.add_first(song)
        elif choice == "3":
            target = input("Chèn sau bài hát nào: ").strip()
            node = playlist.find(target)
            if node:
                new_song = input("Nhập tên bài hát mới: ").strip()
                playlist.add_after(node, new_song)
            else:
                print("Không tìm thấy bài hát!")
        elif choice == "4":
            print("\n--- Danh sách phát ---")
            for song in playlist:
                print(f"🎵 {song}")
        elif choice == "5":
            break

if __name__ == "__main__":
    main()
```

🤖 Gợi ý vận dụng AI để nâng cấp thư viện

1. **Unit Testing:** Nhờ AI viết một file `test_my_linkedlist.py` sử dụng thư viện chuẩn `unittest` hoặc `pytest` để kiểm tra độ bao phủ (coverage) các trường hợp biên (danh sách rỗng, danh sách có 1 phần tử, chèn/xóa sai vị trí).
2. **Sentinel Nodes (Dummy Head/Tail):** Hỏi AI: *"Làm thế nào để viết lại DoublyLinkedList bằng kỹ thuật Sentinel Node (Node giả) ở đầu và cuối để loại bỏ hoàn toàn các câu lệnh kiểm tra `if self.head is None`?"*
3. **Cài đặt đếm ngược (Reverse Iterator):** Yêu cầu AI hướng dẫn viết phương thức `__reversed__()` để có thể sử dụng hàm `reversed(my_list)` duyệt ngược từ `tail` về `head`.